

Exposición — Autenticación, autorización y Keycloak

Guía de estudio para tu parte de la presentación. Sigue el orden que planteaste. Cada sección da por no sabido lo anterior — leé en orden la primera vez.

Corrección importante antes de arrancar: el realm del proyecto se llama **soa-realm**, no **soa-app**. Está así en **.env.example**, en **app/config.py** y en toda la documentación del repo. Decilo bien en la presentación.

1. Qué es Keycloak (la versión corta, sin vueltas)

Cualquier sistema con usuarios necesita resolver tres problemas: saber **quién** es alguien (autenticación), saber **qué puede hacer** (autorización), y guardar contraseñas/tokens sin que se filtren. Lo "casero" es que cada aplicación tenga su propia tabla de usuarios y su propia lógica de login. Funciona, pero se vuelve un problema cuando tenés varias aplicaciones (en nuestro caso: la API y Grafana) que necesitan saber quién está logueado, porque terminás duplicando esa lógica de seguridad dos veces.

Keycloak es un IAM (Identity and Access Management) open source de Red Hat. Es un servicio aparte —un contenedor Docker más, igual que **db** o **nginx**— al que las aplicaciones le **delegan** la autenticación. En vez de que nuestra API reciba una contraseña y la compare contra su propia tabla, le pregunta a Keycloak "¿este usuario es quién dice ser?", y Keycloak responde con un **token** que lo certifica.

No es la única opción (existen Auth0 como SaaS pago, o Authentik como alternativa open source), pero es gratuito, se despliega con Docker igual que el resto de nuestra infraestructura, y es el estándar de facto en el mundo enterprise.

En nuestro sistema, **dos aplicaciones distintas** delegan su login en el mismo Keycloak: la API (FastAPI) y Grafana. Ese es justamente el caso de uso que justifica tener un IAM centralizado en vez de que cada una resuelva el login por su cuenta.

2. Conceptos base y cómo los aplicamos

Realm

Un **realm** es un espacio aislado dentro de Keycloak: agrupa usuarios, roles, clients y toda la configuración de seguridad. Es la unidad de aislamiento más alta — un usuario de un realm no existe en otro, aunque vivan en la misma instancia de Keycloak. Pensalo como una base de datos separada.

Keycloak viene con un realm **master** por defecto, pero ese es para administrar **al propio Keycloak** (crear otros realms, gestionar la instancia), no para usuarios de aplicación. Por eso creamos uno propio: **soa-realm**. Todo lo que sigue (clients, roles, usuarios) vive adentro de **soa-realm**.

Client

Un **client** es el registro, dentro de un realm, de **una aplicación** que usa a Keycloak para autenticar gente. No es un usuario, es la app misma. Nosotros tenemos **dos clients**:

Client	Representa a	Tipo
<code>soa-client</code>	Nuestro backend FastAPI	Confidential
<code>grafana-client</code>	Grafana	Confidential

La diferencia entre **confidential** y **public** client es si la aplicación puede guardar un secreto de forma segura:

- **Public:** apps que corren en el navegador o el dispositivo del usuario (una SPA, una app móvil), donde cualquiera podría inspeccionar el código y encontrar cualquier secreto que pusieras ahí. No tiene `client secret`.
- **Confidential:** aplicaciones backend, que corren en un servidor que el usuario no puede inspeccionar. Tiene un `client secret` — una "contraseña" que identifica a la aplicación (no a un usuario) ante Keycloak.

Tanto el backend FastAPI como Grafana corren server-side, así que ambos son confidential clients, cada uno con su propio secret guardado únicamente en variables de entorno (`KEYCLOAK_CLIENT_SECRET` y `GRAFANA_KEYCLOAK_CLIENT_SECRET`), nunca en código ni en git.

Configuración de `soa-client` (la vas a explicar en la sección 4, pero la tabla completa es):

Opción	Valor	Por qué
Client authentication	On	Lo convierte en confidential — puede guardar el secret de forma segura
Standard flow	On	Queda disponible, aunque no es el que usamos hoy
Direct access grants	On	Es el que usamos: el backend manda usuario/contraseña directo, sin redirects
Implicit flow	Off	Flujo obsoleto, no aplica
Service accounts roles	On	Necesario para que el backend pueda crear usuarios vía Admin API (sección 3)

Configuración de `grafana-client` (la desarrollás en la sección 5):

Opción	Valor	Por qué
Standard flow	On	Es el flujo que usa Grafana: redirect del navegador a Keycloak
Direct access grants	Off	Grafana nunca debe ver la contraseña del usuario — al revés que la API

User

Un usuario dentro de un realm. Keycloak guarda username, email, password (siempre hasheada, nunca en texto plano), y le asigna un **ID único (UUID)** — ese `sub` (subject) es la pieza clave que conecta la identidad de Keycloak con nuestra propia base de datos: lo guardamos en `persons.keycloak_id`.

Role

Un **role** es una etiqueta de permiso. Hay roles de realm (globales, aplicables a cualquier client) y roles de client (específicos de una app). Nosotros usamos solo **realm roles**, porque los permisos que definimos (poder

cargar datos, poder entrar a Grafana) no son exclusivos de un client en particular, son del sistema completo.

Definimos **tres realm roles**:

Rol	Significado en nuestro sistema
VIEWER	Solo lectura. Puede consultar (GET) pero no cargar nada.
OPERATOR	Uso normal: además de leer, puede cargar fotogramas, registrar personas, correr reconocimiento facial.
ADMIN	Todo lo de OPERATOR , y además es el único rol que puede entrar a Grafana.

Matriz de permisos real, por endpoint:

Endpoint	Acción	ADMIN	OPERATOR	VIEWER
GET /models (S1)	Leer	✓	✓	✓
POST /detections (S2)	Cargar	✓	✓	×
GET /frames/{id} (S3)	Leer	✓	✓	✓
GET /frames/search (S4)	Leer	✓	✓	✓
POST /persons (S5.1)	Cargar	✓	✓	×
GET /persons/{id} (S5.2)	Leer	✓	✓	✓
POST /persons/{id}/embeddings (S5.3)	Cargar	✓	✓	×
POST /face-recognition (S5.4)	Consultar	✓	✓	×
Dashboards de Grafana	Leer	✓ (único)	×	×

Detalle importante para una pregunta tipo "¿y si no tengo ningún rol?": un usuario recién registrado, sin **VIEWER/OPERATOR/ADMIN** asignado, **ya puede leer**. Los endpoints **GET** no exigen ningún rol puntual, solo estar autenticado (tener un token válido). El rol **VIEWER** no hace falta asignarlo para habilitar lectura — existe como etiqueta explícita para alguien a quien **no** querés darle **OPERATOR/ADMIN**. Para poder cargar datos, un admin tiene que asignarle **OPERATOR** a mano.

Por qué ningún rol se asigna por defecto (esto tiene una historia, contala, queda bien): en un primer intento se puso **OPERATOR** como "rol asociado" del rol compuesto automático **default-roles-soa-realm** (Keycloak crea uno de estos por cada realm; cualquier rol que metas ahí se le asigna automáticamente a *todo* usuario nuevo). El problema: ese rol compuesto se asigna a **todos sin excepción** y queda **heredado para siempre** — no se le puede sacar a un usuario en particular. Eso significaba que **VIEWER** nunca era una restricción real: aunque le asignaras **VIEWER** a alguien, seguía teniendo **OPERATOR** heredado del rol por defecto. Se sacó **OPERATOR** de ahí por ese motivo. Hoy: **ningún rol es automático**, un admin asigna manualmente **VIEWER**, **OPERATOR** o **ADMIN** desde la consola de Keycloak (**Users** → usuario → **Role mapping** → **Assign role**). No existe (todavía) un endpoint propio para que un admin cambie el rol de otro usuario desde nuestra API — es 100% manual.

Quién es la fuente de verdad de los roles: Keycloak, siempre. El backend FastAPI **nunca asigna ni modifica roles** — solo los **lee** del JWT para decidir si autoriza o no una request.

3. Cómo se llama a Keycloak — su API y el JWT

Keycloak expone dos familias de endpoints REST que usamos:

El endpoint de token (`/protocol/openid-connect/token`)

```
POST /realms/soa-realm/protocol/openid-connect/token
```

Es el endpoint central: ahí se piden todos los tokens, sea cual sea el flujo (login de usuario, refresh, o autenticación de la app misma). Lo que cambia es el parámetro `grant_type` del body. Estos son los tres `grant_type` que usamos:

- **password** (Direct Access Grant): manda `username` + `password` directo, recibe tokens en la misma respuesta. Es lo que usa nuestro `/auth/login`.
- **refresh_token**: manda un refresh token vigente, recibe un access token nuevo sin pedir contraseña. Es lo que usa `/auth/refresh`.
- **client_credentials** (Client Credentials Grant): la *aplicación* se autentica con su propio `client_id` + `client_secret`, sin usuario humano. Recibe un token que representa "soy la app", no a una persona — lo usamos para que el backend pueda crear usuarios (ver más abajo, **service account**).

La Admin REST API (`/admin/realms/...`)

Además del login, Keycloak expone una API completa para administrar el realm programáticamente: crear usuarios, asignar roles, etc. (`/admin/realms/soa-realm/users`, por ejemplo). Es la que nuestro backend usa para crear usuarios cuando alguien se registra desde nuestro formulario propio, en vez de que un humano los cree a mano desde la consola.

Para poder llamar a la Admin API hace falta estar autenticado como algo con permiso de administración.

Ahí entra el **service account**: cuando activás "Service accounts roles" en un client, Keycloak crea automáticamente un usuario especial (`service-account-soa-client`) que representa **a la aplicación misma**. Le asignamos el rol `manage-users` (que pertenece al client interno `realm-management` — Keycloak expone su propia gestión interna como un client más, con roles granulares: `manage-users`, `manage-clients`, `view-users`, etc.). Aplicamos **principio de mínimo privilegio**: el service account solo tiene el permiso que necesita (crear/editar/borrar usuarios), nada de gestión de clients ni del realm completo. Flujo completo: backend pide un token con `grant_type=client_credentials` → Keycloak responde con un token que tiene `manage-users` → el backend usa ese token como **Authorization: Bearer <token>** para llamar a `POST /admin/realms/soa-realm/users`.

El JWT — por qué es importante

Cuando Keycloak autentica a alguien (por cualquiera de los flujos), no te devuelve "sí, esta persona existe" y nada más: te devuelve **tokens**, y el formato concreto de esos tokens es **JWT** (JSON Web Token).

Un JWT es un string con tres partes separadas por puntos: `header.payload.signature`.

- **Header**: metadata, qué algoritmo de firma se usó (en nuestro caso RS256).

- **Payload:** el contenido real — un conjunto de **claims** (afirmaciones). Los que usamos:
 - **sub**: el UUID del usuario en Keycloak (el que guardamos como **keycloak_id**).
 - **iss** (issuer): quién emitió el token — útil para no aceptar tokens de otro realm.
 - **azp**: para qué client se emitió el token.
 - **realm_access.roles**: la lista de roles del usuario (**["OPERATOR", "default-roles-soa-realm", ...]**).
 - **exp**: timestamp de expiración.
 - **preferred_username, email**.
- **Signature:** una firma criptográfica que prueba que el token no fue alterado y que lo emitió Keycloak (y no alguien que se inventó un JWT a mano).

iss en detalle — qué es, cuándo se valida, para qué sirve

iss no es un nombre ni un ID — es literalmente una **URL en texto**, algo como:

```
https://auth.soagmr.mooo.com/realm/soa-realm
```

(o **http://keycloak:8080/realm/soa-realm** si la llamada se hizo por la red interna de Docker — por eso, como vimos en la sección 4 punto 7, no se hardcodea: se le pregunta a Keycloak cuál es el valor real vía el endpoint de discovery).

Cuándo se valida: en *cada* request a un endpoint protegido, dentro de **get_current_user** (**app/security.py**):

```
claims = jwt.decode(
    token,
    signing_key.key,
    algorithms=["RS256"],
    issuer=_get_issuer(),    # ← acá
    ...
)
```

El parámetro **issuer=** le dice a la librería PyJWT: "además de verificar la firma, fijate que el claim **iss** que viene *adentro* del token sea exactamente igual a este string". Si no coincide, tira un error y la request se rechaza con **401**.

Para qué sirve: es una capa extra de seguridad sobre la firma. Asegura que el token lo emitió *específicamente* **soa-realm** de nuestro Keycloak, y no otro realm dentro de la misma instancia (por ejemplo, si alguien creara un realm de prueba) ni, en un escenario más extremo, un token de otro servidor Keycloak. La firma ya protege bastante por sí sola, pero **iss** es una verificación adicional de "este token es para *este* sistema en particular" — defensa en profundidad, no la única barrera.

Punto clave que hay que entender bien: header y payload están solo en **Base64**, *no cifrados* — cualquiera puede decodificarlos y leer el contenido sin ninguna clave (próbalos en **jwt.io**, o **python3 -c "import jwt; print(jwt.decode(token, options={'verify_signature': False}))"**). Por eso un JWT

nunca debe llevar datos sensibles como contraseñas. Lo que garantiza que no fue falsificado no es que esté "escondido", es la **firma**.

Cómo se verifica esa firma sin tener que preguntarle a Keycloak en cada request — JWKS: Keycloak firma con una clave privada que solo él conoce, pero publica la clave **pública** correspondiente en un endpoint:

```
GET /realms/soa-realm/protocol/openid-connect/certs
```

Esto se llama **JWKS** (JSON Web Key Set). Cualquier aplicación puede descargar esa clave pública una vez, cachearla, y usarla para verificar la firma de cualquier JWT **localmente, sin red**, sabiendo con certeza que si la firma es válida, lo emitió Keycloak. En nuestro backend esto vive en `app/security.py`: se usa `PyJWKClient` con `cache_keys=True`, `lifespan=3600` (cachea 1 hora). Esto significa que **Keycloak no se contacta por red en cada request protegida** — solo la primera vez, o cuando el caché expira.

Tipos de tokens que devuelve Keycloak:

- **Access Token:** el que se manda en cada request protegida como header `Authorization: Bearer <token>`. Vida corta (300 segundos / 5 min en nuestras pruebas) — intencional: si se filtra, el daño está acotado en el tiempo.
- **Refresh Token:** vida más larga (1800 seg / 30 min). Sirve para pedir un access token nuevo sin que el usuario reescriba la contraseña.
- **ID Token:** pensado para que la app sepa quién es el usuario — más relevante en flujos con frontend interactivo. No lo usamos explícitamente porque ya extraemos lo que necesitamos del access token.

4. Por qué configuramos Keycloak así — decisión de "todo desde la API"

Las dos opciones que existían

Para tener login propio (sin que el usuario vea jamás la pantalla de Keycloak) había dos caminos:

- **(A) Direct Access Grant:** un formulario de login propio que manda usuario/contraseña a *nuestro* backend, y el backend reenvía esas credenciales a Keycloak.
- **(B) Authorization Code Flow con tema personalizado:** seguir usando el flujo estándar (redirect a Keycloak), pero "vistiendo" la pantalla de login de Keycloak con HTML/CSS propio para que no se note que es Keycloak.

Se eligió (A) porque el requerimiento fue no usar la interfaz de Keycloak en absoluto, ni siquiera personalizada — queríamos control total del formulario en nuestro propio frontend Vue. La contrapartida, para que quede documentada: el MFA nativo de Keycloak (TOTP, doble factor) viene gratis con el flujo (B) porque pasa por la pantalla de Keycloak; con (A) no — si se quiere agregar después, hay que implementarlo como paso extra propio.

Por qué Direct Access Grant "rompe una regla" de OAuth2, y por qué igual es aceptable

El estándar OAuth2 en general **no recomienda** este flujo, porque implica que tu propia aplicación ve la contraseña real del usuario (aunque sea solo de paso, para reenviarla). El principio que se rompe es "solo el Authorization Server debería ver credenciales". Es aceptable cuando la aplicación es **first-party**: cuando la

misma organización controla tanto el formulario de login como el backend que llama a Keycloak. Ese es exactamente nuestro caso — nosotros somos dueños del frontend Vue y del backend FastAPI, no es una app de terceros.

Qué hubo que configurar que no viene por defecto

Esto es la parte más "técnica" de la presentación — listalo así:

1. **Activar "Direct access grants" en `soa-client`** — viene apagado por defecto en un client nuevo de Keycloak (el flujo recomendado por defecto es el redirect). Hubo que activarlo explícitamente.
2. **Activar "Service accounts roles" en `soa-client`** — para que el backend pueda crear usuarios vía Admin API sin que un humano lo haga a mano. Esto generó automáticamente el usuario `service-account-soa-client`.
3. **Asignarle el rol correcto al service account** — `manage-users` del client interno `realm-management`. (Dato real para mostrar que entendieron el problema: en el primer intento se asignaron por error `create-client` y `manage-clients` —que sirven para gestionar *clients*, no usuarios— y se corrigieron quitándolos y asignando el correcto.)
4. **Sacar `OPERATOR` de los roles por defecto del realm** — ya explicado en la sección 2, porque rompía la posibilidad de restringir con `VIEWER`.
5. **`firstName/lastName` obligatorios al crear usuarios** — el realm tiene activado `VERIFY_PROFILE` (perfil de usuario completo obligatorio). Sin esos dos campos, el alta en Keycloak responde `201` igual, pero el login posterior falla con `"Account is not fully set up"` porque Keycloak detecta el perfil incompleto recién al loguearse, no al crearlo. Hubo que mandarlos siempre desde `POST /admin/realms/soa-realm/users`.
6. **`KC_HOSTNAME + KC_PROXY_HEADERS=xforwarded`** — por defecto Keycloak en modo dev no sabe que está detrás de un proxy (nginx) ni cuál es su dominio público real. Sin esto, el `iss` (issuer) de los tokens quedaría mal armado (por ejemplo, con `http://` en vez de `https://`, o con el nombre interno del contenedor). `KC_HOSTNAME` le dice el dominio público real (`auth.soagmr.mooo.com`), y `KC_PROXY_HEADERS=xforwarded` le dice que confíe en los headers `X-Forwarded-*` que manda nginx para saber que el tráfico original es HTTPS aunque nginx le hable por HTTP puro dentro de la red interna.
7. **Validar el `iss` por discovery, no a mano** — esto es un detalle fino y vale la pena mencionarlo porque demuestra entendimiento real: en vez de hardcodear en el backend la URL esperada del issuer, `app/security.py` le pregunta directamente a Keycloak:

```
GET /realms/soa-realm/.well-known/openid-configuration
```

y usa el campo `issuer` de esa respuesta. **Por qué no se compone a mano:** según `KC_HOSTNAME` y cómo llegó la conexión, Keycloak puede generar un `iss` con un esquema/puerto que no coincide ni con la URL interna (`http://keycloak:8080`) ni con la pública (`https://auth.soagmr.mooo.com`) — por ejemplo, si una llamada no pasó por nginx. Pedirle el dato al propio Keycloak por la misma ruta interna que usamos para pedir tokens garantiza que siempre coincida, sin importar cómo esté configurado el hostname.

8. **No validar `aud`, validar `azp` en su lugar** — Keycloak por defecto pone `aud: "account"` en el access token (genérico, no específico de nuestro client). Si validáramos `aud` estrictamente, fallaría siempre. En cambio, validamos que `azp` (el client para el que se emitió el token) sea igual a `soa-client` — así

nos aseguramos de que el token fue emitido específicamente para nuestra API, y no por ejemplo para `grafana-client`.

5. Conexión con Grafana

El protocolo: OpenID Connect, Authorization Code Flow

Aquí **no** usamos Direct Access Grant — al revés que con la API. Grafana usa el flujo **OAuth2 / OIDC estándar con redirect**, llamado **Authorization Code Flow** (o "Standard Flow" en la jerga de Keycloak). Pasos:

1. El usuario hace clic en "Sign in with Keycloak" en Grafana.
2. Grafana redirige el navegador a la pantalla de login **de Keycloak** (no a un formulario propio de Grafana).
3. El usuario ingresa sus credenciales **directamente en Keycloak** — Grafana nunca ve la contraseña.
4. Keycloak redirige de vuelta a Grafana con un código temporal de un solo uso.
5. Grafana (server-side) intercambia ese código por los tokens reales, hablándole a Keycloak directamente (no a través del navegador).

Esto es justo al revés de la decisión que tomamos para la API (sección 4): para la API quisimos *no* mostrar la pantalla de Keycloak; para Grafana, en cambio, no nos importa porque Grafana no es "nuestro" frontend, es una herramienta de monitoreo aparte, y así evitamos reimplementar su login.

El intercambio código → tokens, en detalle (con las URLs reales del proyecto)

La parte que más cuesta entender la primera vez es el paso 4-5 de arriba: "Keycloak redirige con un código, y después Grafana lo cambia por tokens". Desglosado con HTTP real:

Actores: tu navegador, el backend de Grafana, Keycloak. La clave de todo esto es **distinguir qué pasos hace el navegador y cuáles hace el servidor de Grafana directamente**, sin que el navegador se entere.

1. **Click en "Sign in with Keycloak"**. Grafana redirige tu navegador a la URL de autorización (`GF_AUTH_GENERIC_OAUTH_AUTH_URL`), con parámetros en la query string:

```
GET https://auth.soagmr.mooo.com/realms/soa-realm/protocol/openid-connect/auth
    ?response_type=code
    &client_id=grafana-client
    &redirect_uri=https://soagmr.mooo.com/grafana/login/generic_oauth
    &scope=openid email profile
    &state=xyz123
```

Esto lo hace el **navegador**, viajando directo a Keycloak. Grafana ya no participa en este paso — solo lo disparó con el redirect.

2. **Pantalla de login — es la de Keycloak, no la de Grafana**. El navegador aterriza en `auth.soagmr.mooo.com` y muestra el formulario de Keycloak. El usuario escribe usuario/contraseña ahí. **Grafana nunca ve esa contraseña** — ni pasa por sus servidores, va directo del navegador a Keycloak.

3. **Keycloak valida las credenciales y redirige de vuelta, pero no manda el token directo — manda un código de un solo uso:**

```
GET https://soagmr.mooo.com/grafana/login/generic_oauth?
code=ABC456&state=xyz123
```

Ese **code** es un string opaco, sin ningún claim adentro, que solo sirve para canjearlo por tokens reales. Es de un solo uso y expira en segundos (típicamente ~60).

4. **El navegador entrega ese código a Grafana** (la URL de arriba apunta a Grafana, no a Keycloak). Hasta acá, todo lo que pasó por el navegador fue: la pantalla de login y un código inútil por sí solo.
5. **El intercambio real ocurre *fuera* del navegador — server-to-server, Grafana backend → Keycloak**, sin que el navegador vea nada de esto:

```
POST http://keycloak:8080/realms/soa-realm/protocol/openid-
connect/token
grant_type=authorization_code
code=ABC456
client_id=grafana-client
client_secret=<GRAFANA_KEYCLOAK_CLIENT_SECRET>
redirect_uri=https://soagmr.mooo.com/grafana/login/generic_oauth
```

Notar que esto usa **GF_AUTH_GENERIC_OAUTH_TOKEN_URL**, la URL **interna** de Docker — porque esta llamada la hace el proceso de Grafana corriendo en el servidor, no el navegador del usuario.

6. **Keycloak responde con los tokens reales** (**access_token**, **id_token**, **refresh_token**) directo a Grafana — nunca aparecen en la URL del navegador, ni quedan en su historial, ni en logs de nginx.
7. Grafana usa ese **access_token** para pedirle el perfil a Keycloak (**GF_AUTH_GENERIC_OAUTH_API_URL**, el endpoint **userinfo**), evalúa el **role_attribute_path** (**contains(realm_access.roles, 'ADMIN')**) y decide si deja entrar al usuario.
8. Si pasa la validación, Grafana crea **su propia sesión** (una cookie de Grafana) — desde ahí ya no le importa más Keycloak hasta que esa sesión expire.

¿Por qué este lío con un código, en vez de que Keycloak mande el token directo en el redirect del paso 3? Porque ese redirect pasa **por el navegador**: queda en el historial, en logs de servidores intermedios, en el header **Referer** si la página de destino carga recursos externos, visible para extensiones del navegador, etc. Si ahí viajara el **access_token** real (esto existía y se llamaba **Implicit Flow** — por eso lo vimos marcado **Off** en la config de **soa-client**, está deprecado por inseguro), cualquiera que pudiera espiar esa URL se robaría una sesión válida.

En cambio, el **code** que viaja por el navegador es **inútil sin el client_secret**, y ese secret nunca toca el navegador — solo lo tiene el backend de Grafana (es un confidential client, sección 2). Aunque alguien interceptara el código, no podría canjearlo por tokens reales sin el secret, y además el código muere al primer uso. Por eso el intercambio real (paso 5) tiene que ser una llamada server-to-server.

Qué se configuró en Keycloak — `grafana-client`

Opción	Valor	Por qué
Standard flow	On	Es el flujo de redirect que necesita
Direct access grants	Off	Grafana nunca debe ver la contraseña directamente
Redirect URI válida	la URL de callback de Grafana	Sin esto, Keycloak rechaza el redirect de vuelta por seguridad (evita que un atacante redirija el código a otro lado)

Qué se configuró en Grafana — variables de entorno (`docker-compose.yml`)

```
GF_AUTH_GENERIC_OAUTH_ENABLED: "true"
GF_AUTH_GENERIC_OAUTH_CLIENT_ID: grafana-client
GF_AUTH_GENERIC_OAUTH_CLIENT_SECRET: ${GRAFANA_KEYCLOAK_CLIENT_SECRET}
GF_AUTH_GENERIC_OAUTH_SCOPES: "openid email profile"
GF_AUTH_GENERIC_OAUTH_AUTH_URL: .../realms/soa-realm/protocol/openid-connect/auth
GF_AUTH_GENERIC_OAUTH_TOKEN_URL: http://keycloak:8080/realms/soa-realm/protocol/openid-connect/token
GF_AUTH_GENERIC_OAUTH_API_URL: http://keycloak:8080/realms/soa-realm/protocol/openid-connect/userinfo
GF_AUTH_GENERIC_OAUTH_ALLOW_SIGN_UP: "true"
GF_AUTH_GENERIC_OAUTH_ROLE_ATTRIBUTE_PATH: "contains(realm_access.roles, 'ADMIN') && 'Admin' || ''"
GF_AUTH_GENERIC_OAUTH_ROLE_ATTRIBUTE_STRICT: "true"
```

Esto es literalmente todo el "código" que existe del lado de Grafana — cero líneas de Python/JS propias. Grafana ya viene, incorporado en su propio binario, con un **cliente OAuth2/OIDC genérico** (lo que ellos llaman provider `generic_oauth`). Usamos la imagen oficial sin tocarla (`grafana/grafana:11.0.0`) — no hay un Dockerfile custom para Grafana como sí hay para nuestra `app`.

Por qué alcanza con variables de entorno y no hace falta programar nada: Grafana se configura normalmente con un archivo `grafana.ini`, organizado en secciones (`[auth.generic_oauth]`, `[server]`, etc.). Soporta una convención para pisar cualquier setting de ese archivo desde una variable de entorno, con el patrón `GF_<SECCIÓN>_<CLAVE>`:

Variable de entorno	Equivale a, en <code>grafana.ini</code>
<code>GF_AUTH_GENERIC_OAUTH_ENABLED=true</code>	<code>[auth.generic_oauth] → enabled = true</code>
<code>GF_AUTH_GENERIC_OAUTH_AUTH_URL=...</code>	<code>[auth.generic_oauth] → auth_url = ...</code>
<code>GF_SERVER_ROOT_URL=...</code>	<code>[server] → root_url = ...</code>

Es la forma "Docker-friendly" de escribir un `grafana.ini` sin tener que montar un archivo de config aparte como volumen. Toda la lógica real del flujo (armar la URL de redirect con sus parámetros, mandar y verificar el `state`, hacer el `POST` server-to-server con el `code` por el endpoint de token, parsear la respuesta, evaluar

`role_attribute_path`, crear la sesión y la cookie) **ya está programada dentro del binario de Grafana**. Nosotros solo le decimos *con qué Identity Provider hablar* (las URLs de Keycloak) y *con qué reglas* (scopes, mapeo de rol) — no escribimos ni una línea de la lógica del protocolo en sí.

Contraste que vale la pena remarcar en la presentación, porque muestra el porqué de cada decisión de diseño:

- **Grafana → Keycloak**: 100% configuración. Cero código propio, porque Grafana ya trae un cliente OIDC genérico incorporado.
- **Nuestra API (FastAPI) → Keycloak**: ahí **sí escribimos código real** (`app/security.py`, `app/business/keycloak_service.py`, secciones 3 y 6), porque FastAPI es un framework liviano sin cliente OIDC incorporado — tuvimos que programar nosotros mismos la descarga del JWKS, la verificación de firma, la validación de `iss/azp`, y las llamadas a los endpoints de token y de Admin API.

Es la misma lógica, en espejo, de por qué con Grafana usamos Authorization Code Flow (su cliente OIDC integrado ya lo soporta out-of-the-box) y con nuestra API usamos Direct Access Grant con código propio (ahí decidimos nosotros cómo se ve el login, no un cliente genérico de terceros).

Detalle no obvio, muy bueno para mencionar: hay **dos URLs distintas para Keycloak** en esa config, y no es casualidad:

- `AUTH_URL` usa `${KEYCLOAK_PUBLIC_URL}` (`https://auth.soagmr.mooo.com`) porque esa URL la pega el **navegador del usuario** (es el redirect del paso 2) — tiene que ser la dirección pública, accesible desde internet.
- `TOKEN_URL` y `API_URL` usan `http://keycloak:8080` (la dirección **interna** de Docker) porque esas las pega **Grafana server-side** (pasos 4-5) — no necesita salir a internet, va directo por la red interna `soa_net`, más rápido y sin depender de DNS/TLS externos.

Cómo se restringe el login solo a ADMIN — esta es la parte más interesante de toda la integración con Grafana:

- `ROLE_ATTRIBUTE_PATH` es una expresión (lenguaje JMESPath) que Grafana evalúa sobre los claims del token que le devuelve Keycloak. La expresión dice: "si `realm_access.roles` contiene '`ADMIN`', el rol de Grafana es '`Admin`'; si no, es '' (string vacío, sin rol)".
- `ROLE_ATTRIBUTE_STRICT`: "`true`" es la pieza que lo hace una restricción real y no solo una sugerencia: con `strict=true`, si la expresión de arriba da un rol **inválido** o vacío, Grafana **rechaza el login completo** (no entra "sin permisos", directamente no entra). Sin `strict`, Grafana dejaría entrar a cualquiera con un rol por defecto, aunque no sea `ADMIN`.

Resultado: solo alguien con el rol `ADMIN` en Keycloak puede loguearse en Grafana. `OPERATOR` y `VIEWER` ni siquiera pueden entrar a ver el botón de login funcionar — Grafana los rechaza apenas Keycloak les devuelve el token.

Otro detalle de infraestructura, relacionado: `GF_SERVER_PROTOCOL`: `http` — nginx termina el TLS (HTTPS) y le habla a Grafana por HTTP puro dentro de la red interna; sin decirle explícitamente a Grafana que su protocolo "real" es HTTP, entra en un loop de redirect infinito (`ERR_TOO_MANY_REDIRECTS`) porque cree que debería estar en HTTPS y nunca lo está desde su propio punto de vista.

OIDC — ¿es lo mismo que "Iniciar sesión con Google"?

Sí, exactamente — es el ejemplo más cotidiano que existe, y conviene usarlo como analogía en la presentación: **cuando una app usa "Sign in with Google", esa app es el `client`, y Google juega exactamente el mismo papel que Keycloak juega para nosotros: es el Identity Provider** (también llamado OpenID Provider, o el "Authorization Server" en términos de OAuth2). Google muestra su propia pantalla de login (igual que el paso 2 del flujo de arriba), autentica al usuario, y le devuelve a la app un token que dice quién es — mismo mecanismo, mismos pasos, mismo Authorization Code Flow que acabamos de detallar con Grafana.

Para que quede clarísima la diferencia entre los dos estándares mencionados hasta ahora:

- **OAuth 2.0** resuelve *autorización*: "esta app tiene permiso para hacer X en mi nombre" (ej. "esta app puede leer mis archivos de Google Drive"). Por sí solo, **no dice quién es el usuario** — solo da permiso de acceso a un recurso.
- **OIDC** es una capa que se construye *encima* de OAuth2 y agrega *identidad*: además del token de acceso, el Authorization Server también emite un **ID Token** (un JWT firmado) que dice explícitamente "este usuario es Juan Pérez, su email es juan@mail.com". Es lo que hace posible un botón de login real, no solo "dar permiso a un recurso".

Un matiz para mencionar si preguntan, porque demuestra que se entendió bien la diferencia: no todos los "Sign in with X" del mercado son OIDC completo en sentido estricto:

- **Google** y **Apple** sí son Identity Providers OIDC completos: emiten un `id_token` firmado, exponen su propio endpoint de discovery (<https://accounts.google.com/.well-known/openid-configuration>, igual al que usamos nosotros con Keycloak en la sección 4), y la validación es idéntica a la que armamos en `app/security.py`.
- **GitHub**, en su flujo clásico de "Sign in with GitHub", usa **solo OAuth2**, sin la capa OIDC formal — no emite un `id_token` JWT; las apps consiguen el perfil llamando aparte a `GET /user` de su API REST con el access token. Funciona parecido desde afuera, pero no es técnicamente OIDC.

En nuestro proyecto, **Keycloak hace de "Google" propio**: en vez de delegar la identidad a un proveedor externo (lo cual implicaría que todo usuario necesite ya tener una cuenta de Google/GitHub para entrar a nuestro sistema), montamos nuestro propio Identity Provider, con nuestra propia base de usuarios, pero hablando el mismo protocolo estándar (OIDC) que hablan Google o Apple. Esa es la ventaja real de que sea un estándar: Grafana no tuvo que escribir código especial para "hablar con Keycloak" — usa su plugin genérico de OAuth2/OIDC (`generic_oauth`), el mismo que usaría para conectarse a Google, Okta, Auth0, o cualquier otro proveedor que respete el estándar.

6. La API — endpoints, flujos completos, y qué hace el frontend

Mapa de código — qué archivo hace qué, y quién llama a quién

Antes de entrar a los flujos, conviene tener clara la arquitectura en capas (la misma separación que usa el resto del backend: `controllers` → `business` → `repositories`, equivalente a `@RestController` / `@Service` / `@Repository` de Spring). Para auth, son **cinco archivos** y cada uno tiene una responsabilidad bien acotada:

```
app/config.py                (configuración)
  ↑ lo lee
```

```

app/business/keycloak_service.py (habla HTTP con Keycloak)
    ↑ lo usa
app/business/auth_service.py      (orquesta lógica de negocio)
    ↑ lo usa
app/controllers/auth.py           (expone los 3 endpoints públicos)
    ↑ usa los dtos de
app/dtos/auth.py                  (forma de entrada/salida)

app/security.py                   (protege TODOS los demás endpoints – S1 a
S5 – no auth.py)

```

app/config.py — la **Settings** (pydantic-settings) que centraliza las variables de entorno: **keycloak_url**, **keycloak_realm**, **keycloak_client_id**, **keycloak_client_secret**. La instancia **settings** se importa tanto en **keycloak_service.py** como en **security.py** — es la única fuente de estos valores, nadie hardcodea una URL de Keycloak en otro lado.

app/business/keycloak_service.py — es la única pieza del código que habla HTTP directamente con la API de Keycloak (la de la sección 3). Es una clase (**KeycloakService**, instanciada una sola vez como singleton al final del archivo: **keycloak_service = KeycloakService()**) con un método por cada operación que necesitamos:

Método	Qué hace	Quién lo llama
login(email, password)	grant_type=password (Direct Access Grant) contra el endpoint de token	auth_service.login()
refresh(refresh_token)	grant_type=refresh_token contra el endpoint de token	auth_service.refresh()
_get_service_token()	grant_type=client_credentials — token que representa a la app, no a un usuario	Internamente, por create_user() y delete_user()
create_user(email, password, first_name, last_name)	POST /admin/realms/soa-realm/users con el token de servicio	auth_service.register()
delete_user(keycloak_id)	DELETE /admin/realms/soa-realm/users/{id} con el token de servicio	auth_service.register() , solo como compensación si falla el insert local

Este archivo no sabe nada de nuestra base de datos (**persons**) ni de DTOs — solo sabe hablar con Keycloak. Esa separación es a propósito: si el día de mañana cambiamos de IAM (por ejemplo, a Auth0), en teoría solo habría que reescribir este archivo.

app/business/auth_service.py — la capa de orquestación: combina **keycloak_service** con nuestra propia base de datos (**app.repositories.person_repository.PersonRepository**). Tres funciones, una por flujo:

- `register(...)`: primero valida localmente que el email no exista (consulta directa a `persons`, sin tocar Keycloak todavía); si está libre, llama a `keycloak_service.create_user()`; con el UUID que devuelve, hace `repo.create(...)` para insertar en `persons`. Si ese insert falla, hace `db.rollback()` y llama a `keycloak_service.delete_user()` para revertir la creación remota — es la compensación que vimos en la sección 6 más abajo.
- `login(db, email, password)`: llama a `keycloak_service.login()`, decodifica (sin verificar firma — confía en la respuesta directa que él mismo originó) el `access_token` para sacar el `sub`, y busca en `persons` el registro con ese `keycloak_id`.
- `refresh(refresh_token)`: wrapper directo sobre `keycloak_service.refresh()`, sin lógica extra.

Este archivo es el único que conoce **ambos mundos** a la vez (Keycloak y nuestra base de negocio) — es donde vive la lógica de "qué hacer cuando un sistema responde pero el otro falla".

`app/controllers/auth.py` — el router de FastAPI (`prefix="/auth"`) con los 3 endpoints HTTP. Es la capa más delgada: recibe el request, llama a la función correspondiente de `auth_service`, y traduce las excepciones a códigos HTTP (`ValueError` → `401/409` según el caso, `RuntimeError` → `503` si Keycloak está caído). No tiene lógica de negocio propia — todo lo delega.

`app/dtos/auth.py` — los modelos Pydantic que definen la forma exacta de lo que entra y sale de cada endpoint: `RegisterRequest`, `LoginRequest`, `RefreshRequest`, `LoginResponse`, `RefreshResponse`. Sirven para tres cosas a la vez: (1) validación automática del body del request (si falta un campo o el email no tiene formato válido, FastAPI rechaza con `422` antes de que el código del endpoint se ejecute), (2) serialización de la respuesta, y (3) son la fuente de los ejemplos que ves en Swagger (`/api/docs`) — los `json_schema_extra` con `"example": {...}` que tienen `RegisterRequest` y `LoginRequest`.

`app/security.py` — esta es la pieza que **no protege los endpoints de auth**, sino **todos los demás** (S1 a S5). Tiene dos cachés en memoria a nivel de módulo (variables globales, viven mientras el proceso de Uvicorn esté arriba):

- `_jwks_client` (cacheado por `_get_jwks_client()`): un `PyJWKClient` apuntando al endpoint JWKS de Keycloak (`/certs`), con `cache_keys=True`, `lifespan=3600` — cachea las claves públicas por **1 hora**, para no tener que pedírselas a Keycloak en cada request.
- `_issuer` (cacheado por `_get_issuer()`): el valor del `iss` esperado, obtenido una sola vez del endpoint de discovery (`.well-known/openid-configuration`) y guardado para siempre en memoria de proceso (sin TTL — solo se pierde si se reinicia el contenedor `app`).

Con esos dos cachés, `get_current_user(token)` hace la validación completa del JWT (firma + `exp` + `iss` + `azp`) **sin red**, salvo la primera vez. Y `require_role(*roles)` es una capa extra sobre `get_current_user` que además exige un rol específico en `realm_access.roles`.

`get_current_user` y `require_role` se usan como `dependencies=[Depends(...)]` a nivel de cada `APIRouter` en `app/controllers/s1.py`, `s2.py`, `s3.py`, `s5.py` (ver el detalle exacto en la sección 2) — nunca dentro de `auth.py`, porque `/auth/register`, `/auth/login` y `/auth/refresh` son, por definición, los únicos endpoints que tienen que funcionar **sin** tener todavía un token.

Los tres endpoints, en `app/controllers/auth.py`

Endpoint	Qué hace
<code>POST /auth/register</code>	Crea el usuario en Keycloak y en nuestra tabla <code>persons</code>
<code>POST /auth/login</code>	Autentica contra Keycloak (Direct Access Grant), devuelve tokens + perfil
<code>POST /auth/refresh</code>	Cambia un refresh token vigente por un access token nuevo

Además, dos **dependencies** (no son endpoints — son funciones que otros endpoints usan vía `Depends(...)` de FastAPI, se ejecutan antes del código del endpoint):

Dependency	Qué hace
<code>get_current_user</code>	Valida el JWT del request y extrae los claims
<code>require_role(*roles)</code>	Además de lo anterior, exige que el usuario tenga uno de los roles indicados

Flujo de registro, paso a paso

Body: `{ nombre, apellido, email, password, extra? }`.

1. **Validación local primero:** se chequea si ya existe una persona con ese email en nuestra tabla `persons`. Si existe, `409 Conflict` — no tiene sentido llamarle a Keycloak si ya sabemos que va a fallar de nuestro lado.
2. **Token de servicio:** el backend pide un token con `grant_type=client_credentials` (representa "soy la app", no un usuario).
3. **Crear el usuario en Keycloak:** `POST /admin/realms/soa-realm/users` con ese token de servicio, mandando `username`, `email`, `firstName`, `lastName` (obligatorios, ver sección 4 punto 5) y la contraseña (`temporary: false`, para que no tenga que cambiarla en el primer login).
4. Keycloak responde `201` con un header `Location` del cual se extrae el UUID del usuario nuevo (el futuro `keycloak_id`). Automáticamente, ese usuario queda **sin ningún rol** (ver sección 2 — ya no hay rol por defecto).
5. **Insert local:** `INSERT INTO persons (... , keycloak_id)` con el UUID obtenido.
6. **Compensación si falla el paso 5** (ej. error de base de datos): como son dos sistemas independientes, no hay una transacción que abarque a ambos. Si el insert local falla *después* de que Keycloak ya creó el usuario, quedaría un usuario huérfano en Keycloak. La solución: el backend hace `db.rollback()` y llama a `DELETE /admin/realms/soa-realm/users/{id}` para revertir la creación remota antes de propagar el error. Esto es un patrón de **compensación** (parecido a un saga en sistemas distribuidos): no hay rollback automático entre dos sistemas distintos, así que se programa la reversión a mano.
7. Se devuelve `201` con los datos de la persona — **sin tokens**. Registrarse no loguea automáticamente; hay que llamar a `/auth/login` por separado.

Flujo de login, paso a paso

Body: `{ email, password }`.

1. El backend reenvía esas credenciales a Keycloak con `grant_type=password` (Direct Access Grant).

2. Si Keycloak responde **400/401** (credenciales incorrectas), el backend traduce eso a un **401** propio, sin filtrar detalles internos de Keycloak hacia el cliente.
3. Si responde **200**, el backend recibe **access_token** + **refresh_token** + **expires_in**.
4. El backend **decodifica** (no verifica firma — confía en la respuesta directa de una llamada que él mismo inició) el **access_token** para sacar el claim **sub**.
5. Busca en **persons** el registro con **keycloak_id = sub**, para devolver el perfil de negocio (nombre, etc.) en la misma respuesta.
6. Devuelve al frontend: **{ access_token, refresh_token, expires_in, person }**.

Qué guarda el frontend, y qué contiene el token

El frontend Vue usa un store de Pinia (**frontend/src/stores/auth.js**). Al loguearse, guarda en **localStorage**:

- **access_token**
- **refresh_token**
- **person** (el perfil, como JSON)

Cada request subsiguiente (**frontend/src/services/api.js**) lee el **access_token** del store y lo manda como header **Authorization: Bearer <token>**.

Qué hay adentro del access_token (ya lo vimos en la sección 3, repasado en contexto de login): **sub, iss, azp, realm_access.roles, exp, email, preferred_username**. El frontend en sí **no necesita decodificar el token** para nada — solo lo reenvía. Quien lo decodifica y valida es el backend, en cada request protegida.

Cuándo se dispara el refresh, y cómo

No hay un timer en el frontend que cuente los 5 minutos del access token. El mecanismo es **reactivo, no proactivo**: en **api.js**, cada request mira la respuesta —

```
if (res.status === 401 && authStore.refreshToken) {  
  const refreshed = await authStore.tryRefresh()  
  if (refreshed) {  
    // reintentar la request original con el token nuevo  
  }  
}
```

Es decir: el frontend manda la request con el access token que tiene. Si el backend responde **401** (porque el token venció, o porque es inválido) **y** todavía hay un refresh token guardado, automáticamente llama a **POST /auth/refresh** con ese refresh token, guarda el access token nuevo, y **reintenta la request original una sola vez** con el token fresco. Si el refresh también falla (refresh token vencido o inválido), **tryRefresh()** hace **logout()** — limpia todo de **localStorage** y el usuario queda deslogueado, tiene que volver a poner usuario/contraseña.

Del lado del backend (**POST /auth/refresh**), el flujo es simple: reenvía el **refresh_token** a Keycloak con **grant_type=refresh_token**. Si todavía es válido, Keycloak devuelve un access token nuevo (y

normalmente un refresh token nuevo también, que reemplaza al anterior). Si ya venció, Keycloak responde error y el backend traduce eso a **401**.

Y si el refresh token también vence, ¿hay que loguearse de nuevo?

Sí, no hay forma de "recuperarlo" — el usuario tiene que volver a poner email/contraseña. Encadenando lo que ya vimos:

1. El frontend manda **POST /auth/refresh** con el **refresh_token** vencido.
2. Keycloak responde **400/401**.
3. **keycloak_service.refresh()** traduce eso a **ValueError("Refresh token inválido o expirado")** → el controller lo convierte en **401**.
4. En el frontend, **tryRefresh()** ve **!res.ok** y llama a **logout()** — borra **access_token**, **refresh_token** y **person** de **localStorage**, y devuelve **false**.
5. Como **tryRefresh()** devolvió **false**, **api.js** no reintenta nada — el usuario queda deslogueado y la UI lo manda de vuelta a la pantalla de login.

Matiz importante para la presentación: en nuestras pruebas el refresh token dura 30 minutos contra los 5 minutos del access token. Pero mientras el usuario sigue activo *dentro* de esa ventana de 30 min, cada refresh exitoso le devuelve un refresh token **nuevo** (Keycloak rota el refresh token en cada uso) — la sesión se va "estirando" sola mientras hay actividad seguida. El problema aparece solo si pasan más de 30 minutos **sin ningún refresh** (pestaña cerrada, inactividad total): ahí ese refresh token específico ya no sirve, y no queda otra que loguearse de nuevo con la contraseña.

Flujo de una request protegida — qué pasa exactamente con **get_current_user**

Para que quede clarísimo el "por dentro" de cada request a S1-S5:

1. Llega el request con **Authorization: Bearer <token>**.
2. **get_current_user** (en **app/security.py**) se ejecuta antes del código del endpoint: a. Si no viene el header, **401**. b. Descarga (o usa del caché) la clave pública vía JWKS. c. Verifica la firma del JWT con esa clave. Si no coincide, **401**. d. Verifica **exp** — si venció, **401** (acá el frontend dispara el refresh, como vimos arriba). e. Verifica que **iss** coincida con el issuer real de **soa-realm** (vía discovery, sección 4 punto 7). f. Verifica que **azp** sea **soa-client** (sección 4 punto 8) — rechaza tokens emitidos para otro client, como **grafana-client**.
3. Si el endpoint además usa **require_role("OPERATOR", "ADMIN")**, se chequea que alguno de esos roles esté en **realm_access.roles** del token — si no, **403**.
4. Si todo pasó, el endpoint se ejecuta normalmente.

Punto para resaltar en la presentación: en ningún paso de la validación de una request protegida el backend vuelve a llamarle a Keycloak por red — toda la verificación es criptográfica y local (firma + clave pública cacheada). Keycloak solo se contacta por red en: login, registro, refresh, y la primera vez que se necesita el JWKS (o cuando el caché de 1 hora expira). Esto es clave para entender por qué el sistema escala bien: no hay un round-trip a Keycloak en cada request de la API.

7. Resumen visual de los tres flujos

REGISTRO

Usuario → [form Vue] → Backend → (Client Credentials) → Keycloak: crea usuario → UUID

→ Backend: INSERT persons (... , keycloak_id = UUID)

LOGIN

Usuario → [form Vue] → Backend → (Direct Access Grant) → Keycloak: valida user/pass → tokens

→ Backend: busca persons WHERE keycloak_id = sub(token)

→ Backend → Frontend: { access_token, refresh_token, person }

→ Frontend guarda todo en localStorage

REQUEST PROTEGIDA

Frontend → Backend (Authorization: Bearer <access_token>)

→ Backend valida JWT localmente (firma + exp + iss + azp, JWKS cacheado)

→ si falta rol: 403 / si vencido o inválido: 401

→ si 401 y hay refresh_token: frontend llama /auth/refresh y reintenta una vez

LOGIN EN GRAFANA (aparte, no toca nuestra API)

Usuario → Grafana → redirect → Keycloak (pantalla propia de Keycloak) → login

→ Keycloak redirige a Grafana con código → Grafana intercambia código por tokens

→ Grafana chequea realm_access.roles contiene 'ADMIN' (sino, rechaza el login)

8. Preguntas típicas que te puede hacer el profesor (y la respuesta corta)

- **¿Por qué dos bases de datos de Postgres separadas (db y keycloak_db)?** Para no mezclar el esquema interno de Keycloak (decenas de tablas: **USER_ENTITY**, **REALM**, **CREDENTIAL**, etc.) con las tablas de negocio (**persons**, **frames**). Aísla credenciales y simplifica backups/upgrades.
- **¿Por qué Direct Access Grant si OAuth2 lo desrecomienda?** Porque somos first-party: controlamos tanto el frontend como el backend. Ver sección 4.
- **¿Qué pasa si se filtra un access token?** Tiene vida corta (5 min) — el daño está acotado. El refresh token tiene más alcance (30 min) pero nunca se manda a recursos protegidos, solo al endpoint de refresh.
- **¿Por qué no verificás aud?** Porque Keycloak pone **aud**: "account" genérico por defecto; usamos **azp** (a qué client se emitió) en su lugar, que sí es específico.
- **¿Qué pasa si Keycloak se cae?** **/auth/login** y **/auth/register** devuelven **503** (capturado explícitamente en **keycloak_service.py** ante **ConnectionError**). Las requests a endpoints ya protegidos con un token vigente siguen funcionando, porque la validación es local (JWKS cacheado), no depende de que Keycloak esté arriba en ese momento.

- ¿Cómo se le saca el rol **ADMIN** a alguien? Manualmente, consola de Keycloak → **Users** → usuario → **Role mapping** → **Unassign**. No hay endpoint propio para esto todavía.

9. Infraestructura — keycloak y keycloak_db en docker-compose.yml

Antes de leer esto, una aclaración sobre la sintaxis que vas a ver en casi todas las variables: `${VARIABLE:-default}` es sintaxis de Docker Compose, no de Keycloak. Significa "usá el valor de `VARIABLE` si está definida en el `.env`; si no existe, usá lo que sigue después de `:-`". Es lo que permite que el proyecto arranque con valores razonables aunque todavía no exista un `.env` real (pasa en desarrollo local), pero que en producción se puedan pisar con contraseñas fuertes sin tocar el `docker-compose.yml`.

keycloak_db — la base de datos propia de Keycloak

```
keycloak_db:
  image: postgres:16-alpine
  container_name: soa_keycloak_db
  restart: unless-stopped
  environment:
    POSTGRES_USER: ${KEYCLOAK_DB_USER:-keycloak}
    POSTGRES_PASSWORD: ${KEYCLOAK_DB_PASSWORD:-keycloak}
    POSTGRES_DB: ${KEYCLOAK_DB_NAME:-keycloak}
  volumes:
    - keycloak_pgdata:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${KEYCLOAK_DB_USER:-keycloak} -d ${KEYCLOAK_DB_NAME:-keycloak}"]
    interval: 5s
    timeout: 5s
    retries: 10
  networks:
    - soa_net
```

Línea	Qué es
<code>image: postgres:16-alpine</code>	Postgres "liviano" (variante alpine , imagen más chica). A diferencia de la base de la app (pgvector/pgvector:pg16), no necesita la extensión pgvector — Keycloak solo guarda sus propias tablas relacionales (USER_ENTITY , REALM , CLIENT , CREDENTIAL , etc.), nunca vectores. Por eso es una imagen distinta y más simple.
<code>POSTGRES_USER / POSTGRES_PASSWORD / POSTGRES_DB</code>	Las tres variables estándar que la imagen oficial de Postgres usa para crear, en el primer arranque, un usuario, una contraseña y una base de datos vacía. Estas mismas credenciales las

Línea	Qué es
	vuelve a usar keycloak (servicio de abajo) para conectarse — tienen que coincidir, por eso ambos servicios leen las mismas variables KEYCLOAK_DB_* del .env .
volumes: keycloak_pgdata:/var/lib/postgresql/data	Volumen nombrado (declarado al final del compose , junto a pgdata , influxdata , etc.) — sin esto, cada vez que se recrea el contenedor (docker compose up --build) se perdería toda la base de Keycloak: usuarios, roles, todo. Con el volumen, los datos viven en el disco del host, independientes del ciclo de vida del contenedor.
healthcheck con pg_isready	Le dice a Docker Compose cómo verificar que Postgres ya está listo para aceptar conexiones , no solo que el proceso arrancó. pg_isready es un comando que viene con Postgres, devuelve éxito solo cuando el servidor acepta conexiones reales. Se reintenta cada 5 segundos, hasta 10 veces.
networks: soa_net	Solo vive en la red interna — no tiene ports: , confirmando lo que vimos en una pregunta anterior: nadie fuera de los contenedores puede llegarle directo.

Por qué importa el healthcheck en particular: el servicio **keycloak** de abajo tiene **depends_on:** **keycloak_db: condition: service_healthy**. Sin el healthcheck, Compose solo esperaría a que el *proceso* de Postgres arrancara (lo cual pasa rápido), pero Postgres puede tardar un poco más en estar realmente listo para aceptar queries — si Keycloak intentara conectarse en ese instante, fallaría al arrancar. **condition: service_healthy** hace que Compose espere a que el **healthcheck** pase, no solo a que el contenedor exista.

keycloak — el servicio de Keycloak en sí

```
keycloak:
  image: quay.io/keycloak/keycloak:latest
  container_name: soa_keycloak
  restart: unless-stopped
  command: start-dev
  environment:
    KC_DB: postgres
    KC_DB_URL: jdbc:postgresql://keycloak_db:5432/${KEYCLOAK_DB_NAME:-keycloak}
    KC_DB_USERNAME: ${KEYCLOAK_DB_USER:-keycloak}
    KC_DB_PASSWORD: ${KEYCLOAK_DB_PASSWORD:-keycloak}
    KEYCLOAK_ADMIN: ${KEYCLOAK_ADMIN_USER:-admin}
```



```

KEYCLOAK_ADMIN_PASSWORD: ${KEYCLOAK_ADMIN_PASSWORD:-admin}
KC_HOSTNAME: ${KEYCLOAK_HOSTNAME:-localhost}
KC_HOSTNAME_STRICT: "false"
KC_HTTP_ENABLED: "true"
KC_PROXY_HEADERS: xforwarded
depends_on:
  keycloak_db:
    condition: service_healthy
networks:
  - soa_net

```

Variable	Qué hace
<code>image:</code> <code>quay.io/keycloak/keycloak:latest</code>	Imagen oficial, distribuida por Red Hat en su propio registry (quay.io), no en Docker Hub.
<code>command: start-dev</code>	Le dice a Keycloak que arranque en modo desarrollo. Es la diferencia más importante de toda la config: <code>start-dev</code> no exige HTTPS ni un hostname fijo configurado de antemano, ideal para <code>localhost</code> y para no bloquear el desarrollo. El modo de producción (<code>start</code>, sin <code>-dev</code>) exige configurar TLS y hostname explícitamente <i>dentro</i> del propio Keycloak — acá se decidió no migrar a <code>start</code> todavía porque TLS ya lo resuelve nginx por delante, y agregar una segunda capa de TLS adentro de Keycloak sería redundante y más riesgoso sin poder probarlo primero.
<code>KC_DB: postgres</code>	Le dice a Keycloak qué motor de base de datos usar. Sin esto, en modo dev Keycloak usa una base H2 embebida (en memoria/disco local del contenedor) — funciona, pero es efímera y no aguanta reinicios ni múltiples instancias. Con <code>postgres</code>, usa la base real (<code>keycloak_db</code>) con persistencia vía el volumen <code>keycloak_pgdata</code>.
<code>KC_DB_URL</code>	El connection string JDBC. <code>keycloak_db</code> ahí no es una IP — es el nombre del servicio, que Docker resuelve por su DNS interno a la IP real del contenedor <code>keycloak_db</code> dentro de <code>soa_net</code> (el mismo mecanismo que vimos para <code>app</code> → <code>db</code> o <code>nginx</code> → <code>adminer</code>).
<code>KC_DB_USERNAME / KC_DB_PASSWORD</code>	Tienen que ser exactamente las mismas credenciales que <code>POSTGRES_USER/POSTGRES_PASSWORD</code> del servicio <code>keycloak_db</code> — por eso ambos leen las mismas variables <code>KEYCLOAK_DB_*</code>. Si no coinciden, Keycloak no puede autenticarse contra su propia base y no arranca.
<code>KEYCLOAK_ADMIN /</code> <code>KEYCLOAK_ADMIN_PASSWORD</code>	Crean, solo la primera vez que arranca (cuando la base está vacía), un usuario administrador en el realm <code>master</code> — el que se usa para entrar a la consola de administración

Variable	Qué hace
	de Keycloak (<code>/admin/master/console</code>) y desde ahí crear <code>soa-realm</code> , los clients, los roles, etc. No tiene nada que ver con los usuarios de nuestra aplicación (los de <code>soa-realm</code>) — es el admin de Keycloak en sí.
<code>KC_HOSTNAME</code>	El dominio público que Keycloak usa para generar URLs y el <code>issuer</code> de los tokens (ya lo vimos en la sección 4, punto 6) — en local queda <code>localhost</code> , en la VPS es <code>auth.soagmr.mooo.com</code> .
<code>KC_HOSTNAME_STRICT: "false"</code>	Si fuera <code>"true"</code> , Keycloak rechazaría cualquier request cuyo hostname no coincida exactamente con <code>KC_HOSTNAME</code> . En <code>"false"</code> es más permisivo — se eligió así por seguridad ante posibles desajustes durante el primer despliegue (por ejemplo, si una llamada interna llega como <code>keycloak:8080</code> en vez de <code>auth.soagmr.mooo.com</code>), para no bloquear el sistema entero por un desajuste de hostname.
<code>KC_HTTP_ENABLED: "true"</code>	Habilita que Keycloak escuche por HTTP plano (sin TLS) en su puerto interno. Tiene sentido porque el TLS real lo termina nginx por delante (sección de infraestructura) — Keycloak solo necesita hablar HTTP puro hacia dentro de la red Docker; exigirle TLS a Keycloak también sería una capa redundante.
<code>KC_PROXY_HEADERS: xforwarded</code>	Le dice a Keycloak que confíe en los headers <code>X-Forwarded-Proto/X-Forwarded-Host</code> que manda nginx, para que sepa que el tráfico <i>original</i> del usuario fue HTTPS aunque a Keycloak le llegue por HTTP puro desde nginx. Sin esto, Keycloak generaría URLs/issuer con <code>http://</code> en vez de <code>https://</code> , rompiendo la validación de <code>iss</code> que vimos en la sección 3.
<code>depends_on: keycloak_db:</code> <code>condition: service_healthy</code>	Compose no arranca <code>keycloak</code> hasta que el <code>healthcheck</code> de <code>keycloak_db</code> (visto arriba) confirme que Postgres ya acepta conexiones — evita el error de arranque por race condition.
<code>networks: soa_net</code>	Igual que <code>keycloak_db</code> : sin <code>ports:</code> , solo alcanzable dentro de la red interna o vía el subdominio público a través de nginx (recordatorio de una pregunta anterior: se decidió sacarle por completo el <code>ports: 8080:8080</code> que tenía al principio, porque no tenía ningún uso real).